

Project / Team Name

Project: Aadhaar Multi-Integration Platform – In-house Aadhaar Verification Service **Organization:** Pichain Labs **Domain:** Digital onboarding / e-KYC for BFSI & NBFC clients **Dates:** April 2022 – July 2022 **Role:** Backend Developer / Platform Engineer (Aadhaar & KYC Services)

1. Business Opportunity

Aadhaar-based verification was a **business-critical step** in Pichain Labs' onboarding and compliance flows for multiple banking, financial services and NBFC clients. However, the in-house Aadhaar integration project was **non-functional**, and the company depended on third-party API vendors for all Aadhaar verification.

This created two major business risks:

1. **Cost & margin impact** – A significant portion of revenue from Aadhaar-based KYC flows was paid out to external vendors, reducing profit margins for high-volume clients.
2. **Performance & operational risk** – The platform was tightly coupled to vendor SLAs and infrastructure. Any latency or outage on their side immediately impacted customer onboarding journeys and end-user experience.

Given that Aadhaar verification is mandatory in many digital journeys (opening bank accounts, SIM activation, e-sign, ITR/GST filing, etc.), this external dependency was both a **strategic risk** and a **scalability bottleneck**. The system needed to support **10,000+ requests per hour** on regular days and **4–5×** that during peak periods, while remaining cost-efficient.

Two earlier internal attempts to build an in-house Aadhaar platform had failed to reach a stable, production-ready state. Leadership asked **Radheshyam** to take ownership and:

- Revive and stabilise the in-house Aadhaar multi-integration platform.
 - Remove or reduce dependence on external Aadhaar and captcha vendors.
 - Ensure the solution could reliably handle **5–6 lakh requests per hour** during peak loads.
-

2. Radheshyam's Developer Contribution

a) Diagnosing constraints and simplifying the architecture

When Radheshyam took over the project, he analysed the existing Kafka-based producer/consumer architecture and found it **over-engineered and fragile** for an I/O-bound request/response Aadhaar workflow.

- Radheshyam **replaced the Kafka layer with a simpler multi-threaded model**, aligning the design with the actual I/O-heavy workload.
- This reduced operational complexity and made failure scenarios easier to reason about, enabling faster iterations on performance and stability.

b) Building a safe, realistic load-testing environment

Due to regulatory and ethical constraints, the team could not generate bulk traffic against real UIDAI services or use live Aadhaar numbers.

- Radheshyam designed and implemented a **"Dummy Aadhaar Multi-Integration Service"**, which replicated **70–80%** of production behaviour while masking sensitive portions.

- He used **stubs and mocks** for UIDAI interactions, realistic stored captcha images and representative internal KYC data (in S3) to exercise core logic.
- This allowed the team to perform **bulk and stress testing** safely, without violating regulations or overloading UIDAI.

c) Identifying Python GIL limitations and moving to multi-processing

Using custom **K6-based load tests**, Radheshyam:

- Drove the system from **100–1,000+ requests per second**, initially observing good p99 latency (1–2 seconds).
- At higher loads, he noticed unexplained latency spikes and throughput issues.

Through profiling and analysis, Radheshyam identified the **Python Global Interpreter Lock (GIL)** as the bottleneck for the multi-threaded design.

- He **redesigned the concurrency model to use multi-processing** instead of multi-threading, enabling true multi-core utilisation with minimal code changes.
- Subsequent bulk tests showed that the latency spikes disappeared, but long-running soak tests uncovered memory retention and pod crashes, which he then addressed in the next step.

d) Redesigning session management for long-running flows

Each Aadhaar verification required a **10–15 minute live session**. The existing session model stored too much in memory and did not clean up correctly.

- Radheshyam initially experimented with serialising the entire interpreter session using `dill`, but load tests showed very large session files and growing memory usage.
- He refined the approach to store **only per-user session state** in individual files and implemented **explicit cleanup** when a session completed.
- This **per-user session model with deliberate deletion** removed long-term memory buildup and stabilised pods during **multi-day soak tests**.

e) Designing and deploying an in-house captcha engine

A key strategic goal was to remove reliance on captcha vendors, but the team lacked a large labelled dataset.

Radheshyam designed a **two-phase approach**:

1. Bootstrap using a third-party vendor (short-term)

- He integrated a third-party captcha service as a **temporary step** and instrumented the Aadhaar flow so that:
 - Every captcha image + vendor answer + UIDAI response were stored in S3.
 - Correct solutions went to a **“correct” bucket**, rejected ones to a **“wrong” bucket**.
- Within a day, this pipeline collected **1 lakh+ correctly labelled** and **10k+ incorrectly labelled** captchas.

2. Error analysis, ML training and in-house engine

- Radheshyam analysed the **wrongly labelled** captchas and found consistent vendor weaknesses (e.g., characters like 8, 0, z, Z).
- He manually corrected **~3,000 hard samples** and merged them into the training set.
- Using an open-source **Keras model**, he trained and validated an in-house captcha classifier achieving **~85% accuracy** on real UIDAI captchas.

- Radheshyam then packaged this as a **standalone captcha microservice** and integrated it into the Aadhaar pipeline, **fully replacing the third-party vendor**.

f) Adding Redis caching to improve resilience

To handle user retries and UIDAI slowdowns:

- Radheshyam implemented **Redis-based caching**, so repeated submissions for the same candidate within a short time window reused the **last known response** instead of calling UIDAI again.
- This reduced unnecessary upstream load, smoothed traffic spikes and improved perceived reliability during partial outages.

g) Eliminating fragile rotating proxies with a serverless reverse proxy

Over time, UIDAI began **identifying and blocking IPs** from the rotating forward proxy pool (~1,000 proxies), shrinking the pool and causing intermittent connectivity and latency issues.

- Radheshyam investigated this behaviour and proposed replacing the fragile proxy pool with a **serverless reverse proxy** pattern.
- He redesigned the outbound connectivity model, removing the need to manage a static set of forward proxies and providing a more **stable, scalable network path** for UIDAI calls.

h) End-to-end load tests and production rollout

After all architectural and resilience enhancements, Radheshyam:

- Updated K6 scripts to simulate **normal, peak and repeated-request (“burst”) scenarios**.
- Verified that the in-house service could reliably handle **5–6 lakh requests per hour** during peak tests, with no pod crashes or memory leaks.
- Worked with the **product manager and tech lead** to plan a **phased rollout** to clients, support their testing and address feedback during migration from third-party flows.

3. Impact

Business impact

- Radheshyam **converted a failed in-house prototype** into a **fully production-ready Aadhaar verification platform**, enabling Pichain Labs to **replace third-party Aadhaar API and captcha vendors** for high-volume clients.
- This delivered **significant recurring cost savings** per verification, directly improving margins in high-volume BFSI/NBFC KYC journeys.
- Clients reported that the **v2 in-house Aadhaar service was noticeably faster and more stable** than the previous implementation, leading to smoother customer onboarding and increased trust in the platform.

Technical and operational impact

- Radheshyam designed and delivered an end-to-end solution that reliably handles **5–6 lakh requests per hour** at peak, with stable response times and no crashes in multi-day soak tests.
- He **simplified and hardened the architecture** (Kafka → multi-threading → multi-processing + improved session management), removing memory leaks and scaling limitations linked to the Python GIL and poor state handling.
- He created a reusable **Dummy Aadhaar Multi-Integration Service** and **K6 test harness** that the team can now use to validate future changes without touching real Aadhaar data or UIDAI.

- Radheshyam designed and implemented an **in-house captcha engine**, from data collection (1L+ correct, 10k+ incorrect samples) to model training (~85% accuracy) and deployment as a microservice, fully removing the third-party captcha dependency.
- He improved resilience via **Redis caching** and a **serverless reverse proxy network pattern**, reducing failures during UIDAI issues and eliminating fragile rotating proxy pools.

Personal / role impact

- Radheshyam took **end-to-end ownership** of a project that two previous developers could not stabilise, covering architecture, backend engineering, performance testing, ML model training and production rollout.
 - He acted as a **technical problem solver and driver**: identifying root causes (GIL, memory leaks, proxy blocking), proposing and implementing alternatives, validating them with data and leading the technical aspects of the v2 launch.
 - The result is a **strategic in-house capability** that reduces cost, improves performance and gives the organisation full control over the Aadhaar verification stack.
-

4. Lessons Learned

- **Start simple and iterate with data** – Radheshyam learned that removing unnecessary complexity (Kafka) and iterating with measurement (load tests, soak tests) leads to more reliable systems than “big-bang” designs.
 - **Match concurrency model to language/runtime** – The Python GIL forced a shift from threads to processes. Radheshyam now evaluates runtime constraints (GIL, memory model) early in architecture decisions.
 - **Design state and session lifecycle deliberately** – The move from “pickle everything” to **per-user session files with explicit cleanup** reinforced the importance of memory lifecycle design for long-running flows.
 - **Use safe test harnesses for regulated systems** – Building the Dummy Aadhaar service showed Radheshyam the value of good simulators and stubs when real systems cannot be load-tested directly.
 - **Use vendors as a bootstrap, not a crutch** – The temporary captcha vendor + data collection pipeline demonstrated how to **leverage third parties strategically**, then remove dependency via an in-house model.
 - **Network layers can be hidden single points of failure** – The rotating proxy pool experience taught Radheshyam to think about how external systems may block or throttle traffic over time, and to design more robust network patterns (serverless reverse proxy).
 - **Resilience patterns are as important as raw throughput** – Redis caching and graceful handling of UIDAI downtime improved user experience even under partial failure.
 - **Stakeholder communication matters** – Working with the product manager, tech lead and clients during v2 rollout reinforced for Radheshyam that successful delivery is not just code, but also **clear communication, phased adoption and feedback loops**.
-

Skills Demonstrated (Core/Common/Specialist)

Core Skills (Experienced)

- **Problem Solving & Root Cause Analysis** – **Experienced** Radheshyam repeatedly identified deep technical root causes (GIL, memory leaks, proxy blocking) and applied validated fixes using data from systematic load tests.

- **Disciplined Development & Engineering Practices – Experienced** Radheshyam used an experiment-driven approach with automated K6 + Dummy Aadhaar tests to guide every major design change instead of relying on ad hoc debugging.
 - **System Design & Architecture – Experienced** Radheshyam simplified a failed Kafka-based design into a clean, microservice-oriented Aadhaar platform with robust session handling, captcha engine and proxy layer.
 - **Performance & Resilience Engineering – Experienced** Radheshyam engineered the system to handle 5–6 lakh requests/hour using multi-processing, efficient session cleanup, Redis caching and resilient outbound connectivity.
 - **Test Strategy & Automation (Load/Soak/Failure Testing) – Experienced** Radheshyam built and used a realistic automated test harness (Dummy Aadhaar + K6) for bulk, spike and soak testing under production-like scenarios.
-

Common Skills (Foundation)

- **Communication & Collaboration – Foundation** Radheshyam worked closely with the product manager, tech lead and clients to plan rollout, support testing and incorporate feedback on performance and stability.
 - **Growth Mindset & Continuous Learning – Foundation** Radheshyam learned and applied new tools and techniques (K6, Redis, serverless reverse proxy, Keras) to solve project-specific challenges end to end.
 - **Curiosity & Exploration – Foundation** Radheshyam deeply investigated vendor captcha failures and proxy blocking patterns, going beyond symptoms to understand and redesign the underlying approach.
 - **Professionalism & Accountability – Foundation** Radheshyam took full ownership of a previously failed project, ensuring regulatory-safe testing and delivering a stable production platform that met business goals.
-

Specialist Skills

- **Backend & Platform Engineering for BFSI/KYC – Experienced** Radheshyam designed and implemented a high-throughput Aadhaar e-KYC platform for BFSI/NBFC clients, replacing third-party services with a reliable in-house solution.
- **Cloud / Microservices & Distributed Systems – Experienced** Radheshyam delivered a containerised, microservice-style architecture (core Aadhaar service, captcha microservice, caching, reverse proxy) that scales reliably under peak load.
- **Data & AI / ML Engineering (Captcha Model) – Foundation–Experienced** Radheshyam built a full captcha ML pipeline—from data collection and labelling to Keras model training and deployment as a production microservice with ~85% accuracy.
- **Security, Compliance & Responsible Use of External Systems – Foundation** Radheshyam ensured all performance testing respected UIDAI and data regulations by using simulated services and synthetic data instead of stressing real government systems.